

```

!pip install -q transformers torch accelerate datasets bitsandbytes
scipy

import torch
import re
import numpy as np
import logging
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM,
BitsAndBytesConfig
from scipy.stats import ttest_rel

logging.getLogger("transformers").setLevel(logging.ERROR)

device = "cuda" if torch.cuda.is_available() else "cpu"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4"
)

model_name = "mistralai/Mistral-7B-Instruct-v0.2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)
model.config.pad_token_id = tokenizer.eos_token_id
model.eval()

dataset = load_dataset("gsm8k", "main",
split="test").shuffle(seed=42).select(range(30))

def extract_gold(answer):
    match = re.search(r"####s*(-?\d+\.\d*)", answer)
    return float(match.group(1)) if match else None

def extract_pred(text):
    nums = re.findall(r"-?\d+\.\d*", text.replace(",", ""))
    return float(nums[-1]) if nums else None

def zero_prompt(q):
    return f"Solve and give only the final numeric answer.\n\n{q}"

```

```

def few_prompt(q, examples):
    prompt = ""
    for ex in examples:
        prompt += f"Q: {ex['question']}\nA: {extract_gold(ex['answer'])}\n\n"
    prompt += f"Q: {q}\nA:"
    return prompt

def cot_prompt(q):
    return f"Solve step by step and end with Answer: <number>\n\nQuestion: {q}\nLet's think step by step."

def generate(prompt):
    inputs = tokenizer(prompt, return_tensors="pt").to(device)
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=128,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id
        )
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

few_examples = dataset.select(range(3))

def evaluate(mode):
    correct = []
    hallucinations = []

    for item in dataset:
        q = item["question"]
        gold = extract_gold(item["answer"])

        if mode == "zero":
            prompt = zero_prompt(q)
        elif mode == "few":
            prompt = few_prompt(q, few_examples)
        else:
            prompt = cot_prompt(q)

        output = generate(prompt)
        pred = extract_pred(output)

        if pred is None:
            hallucinations.append(1)
            correct.append(0)
        else:
            hallucinations.append(0)
            correct.append(int(abs(pred - gold) < 1e-3))

```

```

    return np.array(correct), np.mean(correct),
np.mean(hallucinations)

correct_zero, acc_zero, hall_zero = evaluate("zero")
correct_few, acc_few, hall_few = evaluate("few")
correct_cot, acc_cot, hall_cot = evaluate("cot")

t_zero_cot = ttest_rel(correct_zero, correct_cot)
t_few_cot = ttest_rel(correct_few, correct_cot)

print("Zero-shot Accuracy:", acc_zero)
print("Few-shot Accuracy:", acc_few)
print("CoT Accuracy:", acc_cot)
print("Zero-shot Hallucination:", hall_zero)
print("Few-shot Hallucination:", hall_few)
print("CoT Hallucination:", hall_cot)
print("Zero vs CoT t-test:", t_zero_cot)
print("Few vs CoT t-test:", t_few_cot)

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.

{"model_id": "1987a207a4064dafb56942e514e85c02", "version_major": 2, "version_minor": 0}

Zero-shot Accuracy: 0.13333333333333333
Few-shot Accuracy: 0.03333333333333333
CoT Accuracy: 0.1
Zero-shot Hallucination: 0.0
Few-shot Hallucination: 0.0
CoT Hallucination: 0.0
Zero vs CoT t-test:
TtestResult(statistic=np.float64(0.4411698907260347),
pvalue=np.float64(0.6623617732700398), df=np.int64(29))
Few vs CoT t-test: TtestResult(statistic=np.float64(-1.0),
pvalue=np.float64(0.32558198801619365), df=np.int64(29))

```